

1. Find the Second Highest Salary

Description

Find the employee(s) with the second highest salary from the Employee table.

Scenario

A company wants to identify the second-highest earning employee for a bonus review.

Sample Data

EmpID	Name	Salary
101	John	50000
102	David	70000
103	Smith	90000
104	Alice	70000

SQL Query

```
SELECT *  
FROM Employee  
WHERE Salary = (  
    SELECT MAX(Salary)  
    FROM Employee  
    WHERE Salary < (SELECT MAX(Salary) FROM Employee)  
);
```

Solution Explanation

1. Find the highest salary.
2. Exclude that salary.
3. Find the highest remaining salary.

Expected Output

EmpID	Name	Salary
102	David	70000
104	Alice	70000

2. Find Duplicate Emails

Description

Find all email addresses that appear more than once in the Users table.

Scenario

A system administrator wants to detect duplicate user accounts created with the same email.

SQL Query

```
SELECT Email,  
       COUNT(*) AS Total  
FROM Users  
GROUP BY Email  
HAVING COUNT(*) > 1;
```

Solution Explanation

Group the rows by email and count how many times each email appears. Then filter only those with a count greater than 1.

Expected Output

Email	Total
a@gmail.com	2

3. Customers Without Orders

Description

Find customers who have never placed an order.

Scenario

A business wants to identify inactive customers who have not purchased anything yet.

SQL Query

```
SELECT c.CustomerID,  
       c.CustomerName  
FROM Customers c  
LEFT JOIN Orders o  
ON c.CustomerID = o.CustomerID  
WHERE o.CustomerID IS NULL;
```

Solution Explanation

A LEFT JOIN keeps all customers. If a customer has no matching order, the order columns will be NULL.

Expected Output

CustomerID	CustomerName
5	Rahul
8	Ankit

4. Highest Salary in Each Department

Description

Find the maximum salary in each department.

Scenario

The HR team wants to understand the highest-paid person in every department.

SQL Query

```
SELECT Department,  
       MAX(Salary) AS HighestSalary  
FROM Employee  
GROUP BY Department;
```

Solution Explanation

Use GROUP BY to create one result row per department and MAX() to get the highest salary inside each group.

Expected Output

Department	HighestSalary
HR	75000
IT	120000
Finance	98000

5. Employees Earning More Than Their Manager

Description

Find employees whose salary is greater than their manager's salary.

Scenario

A company wants to check if any employee is earning more than the person supervising them.

SQL Query

```
SELECT e.Name AS Employee,  
       m.Name AS Manager  
FROM Employee e  
JOIN Employee m  
ON e.ManagerID = m.EmpID  
WHERE e.Salary > m.Salary;
```

Solution Explanation

This uses a self-join because the Employee table contains both employee and manager information.

Expected Output

Employee	Manager
John	David

6. Top 3 Highest Paid Employees

Description

Return the top 3 employees with the highest salary.

Scenario

A company wants a leaderboard of its highest-paid employees.

SQL Query

```
SELECT *  
FROM Employee  
ORDER BY Salary DESC  
LIMIT 3;
```

Solution Explanation

Sort salaries from highest to lowest and return the first 3 rows.

Expected Output

Name	Salary
Smith	120000
John	115000
David	110000

7. Departments Having More Than 5 Employees

Description

Find departments with more than 5 employees.

Scenario

The management wants to know which departments are large in size.

SQL Query

```
SELECT Department,  
       COUNT(*) AS Employees  
FROM Employee  
GROUP BY Department  
HAVING COUNT(*) > 5;
```

Solution Explanation

Group records by department, count employees in each group, and filter those above 5.

Expected Output

Department	Employees
IT	8

8. Monthly Sales Report

Description

Create a monthly sales summary.

Scenario

A business wants to monitor sales performance month by month.

SQL Query

```
SELECT DATE_FORMAT(OrderDate, '%Y-%m') AS Month,  
       SUM(Amount) AS TotalSales  
FROM Orders  
GROUP BY DATE_FORMAT(OrderDate, '%Y-%m');
```

Solution Explanation

Extract the year and month from the order date, then aggregate the sales values for each month.

Expected Output

Month	TotalSales
2025-01	250000
2025-02	185000

9. Most Expensive Product

Description

Find the product with the highest price.

Scenario

A store wants to know which product is the most expensive in its catalog.

SQL Query

```
SELECT *  
FROM Product  
ORDER BY Price DESC  
LIMIT 1;
```

Solution Explanation

Sort the products by price in descending order and return the first row.

Expected Output

Product	Price
Laptop	90000

10. Find Missing Employee IDs

Description

Find IDs that are missing in a sequence of employee IDs.

Scenario

A company wants to detect gaps in employee ID numbering.

SQL Query

```
SELECT e1.EmpID + 1 AS MissingID
FROM Employee e1
LEFT JOIN Employee e2
ON e1.EmpID + 1 = e2.EmpID
WHERE e2.EmpID IS NULL;
```

Solution Explanation

Compare each employee ID with the next expected ID and find the ones that do not exist.

Expected Output

MissingID
106

11. Running Total of Sales

Description

Calculate a cumulative total of sales over time.

Scenario

A finance team wants to see the running sales amount day by day.

SQL Query

```
SELECT OrderDate,  
       Amount,  
       SUM(Amount) OVER (ORDER BY OrderDate) AS RunningTotal  
FROM Sales;
```

Solution Explanation

A window function keeps a running total as it processes rows in date order.

Expected Output

Date	Amount	RunningTotal
1-Jan	100	100
2-Jan	200	300
3-Jan	150	450

12. Rank Employees by Salary

Description

Assign a rank to employees based on salary.

Scenario

A company wants to rank employees from highest to lowest salary.

SQL Query

```
SELECT Name,  
       Salary,  
       RANK() OVER (ORDER BY Salary DESC) AS RankNo  
FROM Employee;
```

Solution Explanation

The RANK() window function assigns ranks while handling ties properly.

Expected Output

Name	Salary	RankNo
John	120000	1
David	110000	2

13. Customers Who Ordered Every Product

Description

Find customers who purchased all available products.

Scenario

A retailer wants to identify customers who bought the full product catalog.

SQL Query

```
SELECT CustomerID  
FROM Orders  
GROUP BY CustomerID  
HAVING COUNT(DISTINCT ProductID) = (  
    SELECT COUNT(*)  
    FROM Product  
);
```

Solution Explanation

Count the distinct products purchased by each customer and compare that count to the total number of products in the Product table.

Expected Output

CustomerID
102

14. Latest Order of Each Customer

Description

Find the most recent order for each customer.

Scenario

A business wants to know the latest purchase made by every customer.

SQL Query

```
SELECT *  
FROM (  
    SELECT *,  
        ROW_NUMBER() OVER (  
            PARTITION BY CustomerID  
            ORDER BY OrderDate DESC  
        ) AS rn  
    FROM Orders  
) x  
WHERE rn = 1;
```

Solution Explanation

ROW_NUMBER() assigns a sequence number to each order within each customer group. Taking row 1 gives the latest order.

Expected Output

The query returns the latest order for every customer.

15. Find Consecutive Login Days

Description

Identify the previous login date for each login event.

Scenario

A product team wants to analyze user activity patterns.

SQL Query

```
SELECT UserID,
       LoginDate,
       LAG(LoginDate) OVER (
         PARTITION BY UserID
         ORDER BY LoginDate
       ) AS PreviousDate
FROM LoginHistory;
```

Solution Explanation

The LAG() function compares each row with the previous row in the same user group.

Expected Output

The query shows the earlier login date for each user's login record.

16. Remove Duplicate Records

Description

Delete duplicate rows while keeping one copy.

Scenario

A database contains duplicate email records and needs cleanup.

SQL Query

```
WITH CTE AS (  
    SELECT *,  
           ROW_NUMBER() OVER (  
               PARTITION BY Email  
               ORDER BY ID  
           ) AS rn  
    FROM Users  
)  
DELETE FROM CTE  
WHERE rn > 1;
```

Solution Explanation

ROW_NUMBER() assigns a row number to each duplicate group. Rows with number greater than 1 are removed.

Expected Output

Duplicate email records are removed, leaving one row per email.

17. Employees Joined This Month

Description

Find employees who joined in the current month.

Scenario

HR wants to review employees hired in the current month.

SQL Query

```
SELECT *  
FROM Employee  
WHERE MONTH(HireDate) = MONTH(CURRENT_DATE)  
      AND YEAR(HireDate) = YEAR(CURRENT_DATE);
```

Solution Explanation

Compare the month and year of HireDate with the current date.

Expected Output

Employees hired in the current month are returned.

18. Find Inactive Customers

Description

Find customers who have not placed an order in the last 6 months.

Scenario

A company wants to identify customers who are no longer active.

SQL Query

```
SELECT c.CustomerID
FROM Customers c
LEFT JOIN Orders o
ON c.CustomerID = o.CustomerID
AND o.OrderDate >= CURRENT_DATE - INTERVAL 6 MONTH
WHERE o.CustomerID IS NULL;
```

Solution Explanation

The join checks for recent orders. Customers with no recent order are considered inactive.

Expected Output

Customers who have not ordered in the last 6 months.

19. Find Products Never Sold

Description

Find products that have never been sold.

Scenario

A store wants to identify inventory items that have not yet generated sales.

SQL Query

```
SELECT ProductName
FROM Product p
LEFT JOIN Orders o
ON p.ProductID = o.ProductID
WHERE o.ProductID IS NULL;
```

Solution Explanation

Products with no matching order records are unsold products.

Expected Output

Unsold products are returned.

20. Find Employee with Longest Tenure

Description

Find the employee who has been with the company the longest.

Scenario

The HR team wants to identify the employee with the oldest joining date.

SQL Query

```
SELECT *
FROM Employee
ORDER BY HireDate
LIMIT 1;
```

Solution Explanation

Sort the records by HireDate in ascending order and pick the first row.

Expected Output

Employee	HireDate
John	2015-01-15

Skills Covered

Topic	Questions
GROUP BY & HAVING	2, 4, 7, 13
Joins	3, 5, 18, 19
Subqueries	1, 13
Aggregate Functions	2, 4, 7, 8
Window Functions	11, 12, 14, 15, 16
Ranking	12, 14
CTE	16
Date Functions	8, 17, 18
Self Join	5
NULL Handling	3, 19
Real-World Business Scenarios	All

These questions closely resemble SQL interview questions asked for roles such as SQL Developer, Data Analyst, Data Engineer, and BI Developer.